TABLE OF CONTENTS

S.I NO.	SECTION TOPIC	PAGE NO.
1	**Introduction**	6
2	**Objectives, Scope and Limitations**	7
3	**Operating System Memory Management Concepts**	8
4	**System Architecture and Design**	10
5	**Module Description and Dependencies**	12
6	**Memory Model and Allocation State**	13
7	**First-Fit, Best-Fit and Worst-Fit Algorithms**	14
8	**Fragmentation and Compaction**	15
9	**Memory Allocation Examples and Results**	16
10	**Paging Model and Address Translation**	17
11	**FIFO, LRU and Optimal Replacement**	19
12	**Page Replacement Worked Examples**	20
13	**Paging Results and Comparison**	22
14	**Workloads and Performance Metrics**	23
15	**Benchmarking and Performance Analysis**	24
16	**Testing Strategy and Correctness**	26
17	**Test Cases and Test Results**	27
18	**User Interface and Workflow**	28
19	**Execution Examples and Output**	29
20	**Repository Structure and Development Conventions**	30
21	**Build System and Continuous Integration**	31
22	**Setup Instructions and Documentation**	32
23	**Conclusion and Future Scope**	33
24	**Appendix**	34

ABSTRACT

MemForge is a deterministic simulator of an operating-system memory manager, written in C17 and also compiled to WebAssembly. It covers contiguous allocation (First-, Best- and Worst-Fit, splitting, coalescing, sliding compaction) and demand paging (single-level page tables, address translation, FIFO, LRU and OPT replacement). Policies only decide and engines only mutate, and every change is published as an event, so identical inputs and seeds always produce byte-identical traces. On 30 seeded workloads at 90% utilisation, Best-Fit reaches a 91.4 ± 0.4% success rate against 90.0 ± 0.6% for First-Fit and 87.2 ± 0.7% for Worst-Fit, while First-Fit scans the fewest holes (3.21 vs. 5.24 per request). On 2,000-reference workloads, faults follow OPT < LRU < FIFO at every frame count (103.2, 138.9 and 253.1 at 8 frames), and a frame sweep reproduces Belady's anomaly (9 → 10 faults). All eight acceptance criteria and 17 test suites pass under AddressSanitizer, UBSan and Valgrind.

Keywords—memory management, deterministic simulation, contiguous allocation, fragmentation, demand paging, page replacement, Belady's anomaly.

1. INTRODUCTION

Every running program (a *process*) needs space in the computer's main memory (RAM) to hold its code and data. RAM is limited and shared by many processes, so the operating system must decide *who gets which part of memory*, give it back when a process finishes, and stop processes from touching each other's memory. This job is called *memory management*.

Memory management decides where each process lives in physical memory and how its addresses are translated. Textbooks present contiguous allocation and demand paging with small hand-worked examples, but the trade-offs between policies only become visible when they run on the same workload, step by step, at scale.

MemForge closes that gap. It is a modular C17 simulator with a command-line tool for scripted experiments and a browser laboratory that runs the *same* engine compiled to WebAssembly. Its three design pillars are summarised in Fig. 1; the rest of this paper follows the order in which the system was built: concepts, architecture, the contiguous subsystem, the paging subsystem, experiments, testing, interfaces and engineering.

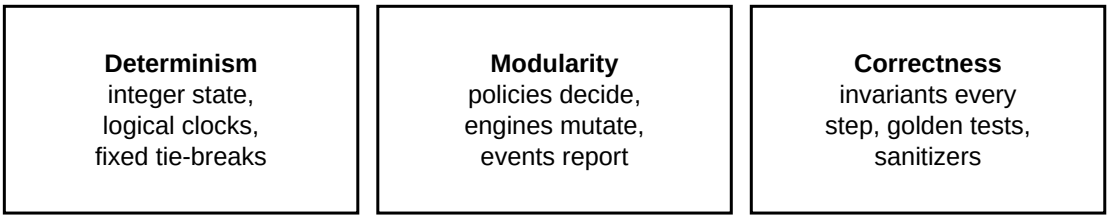


Fig. 1. The three design pillars of MemForge.

2. OBJECTIVES, SCOPE AND LIMITATIONS

2.1 Objectives

1. Implement First-, Best- and Worst-Fit with splitting, coalescing, fragmentation metrics and compaction.
2. Simulate translation, page faults and FIFO, LRU and OPT replacement.
3. Compare policies fairly on identical, reproducible workloads.
4. Verify correctness with invariants, automated tests and memory-safety tools.
5. Offer a CLI and an interactive WebAssembly lab.

2.2 Scope

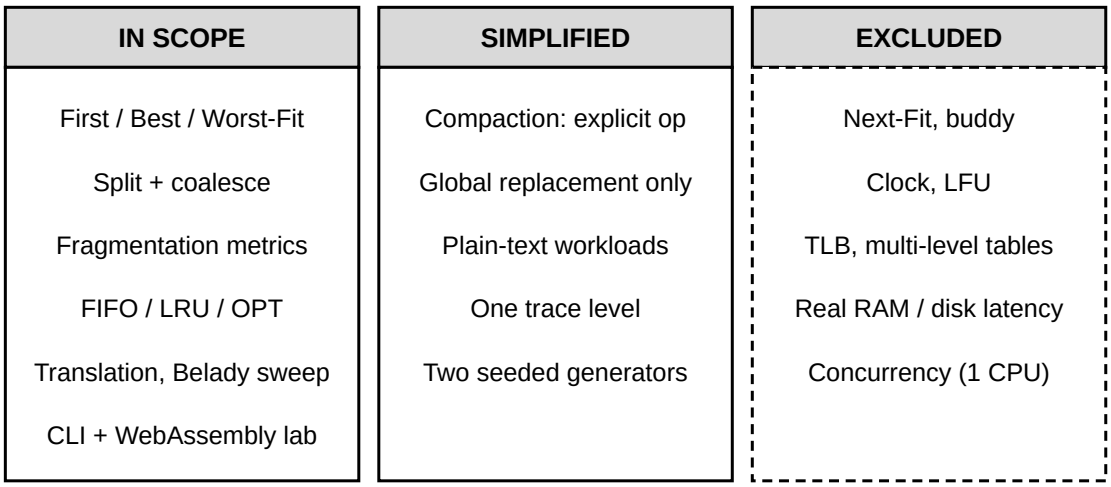


Fig. 2. Scope boundaries: what is implemented, simplified and excluded.

2.3 Limitations

Memory is simulated, not real RAM; one CPU applies one operation at a time; disk I/O is counted but its latency is not modelled; page tables are single-level with no TLB; Next-Fit, Clock and other policies are out; compaction is never automatic.

3. OPERATING SYSTEM MEMORY MANAGEMENT CONCEPTS

From the ground up. Think of memory as one long row of numbered boxes (addresses 0, 1, 2, …). A program asks for, say, 212 boxes; the OS must find 212 *free boxes in a row*, mark them as used, and later mark them free again. Two broad strategies exist: give each request one unbroken run of boxes (*contiguous allocation*), or cut both programs and memory into equal-sized pieces that can be placed anywhere (*paging*). MemForge simulates both.

The MMU maps each *logical* address produced by the CPU to a *physical* address and traps invalid accesses. Under contiguous allocation each request receives one run of memory taken from a free *hole*: First-Fit takes the first hole that fits, Best-Fit the smallest and Worst-Fit the largest. *External* fragmentation is free memory split into holes too small to use; *internal* fragmentation is unused space inside a block (zero in MemForge, since granted = requested). Compaction slides blocks together to merge the holes (Fig. 3).

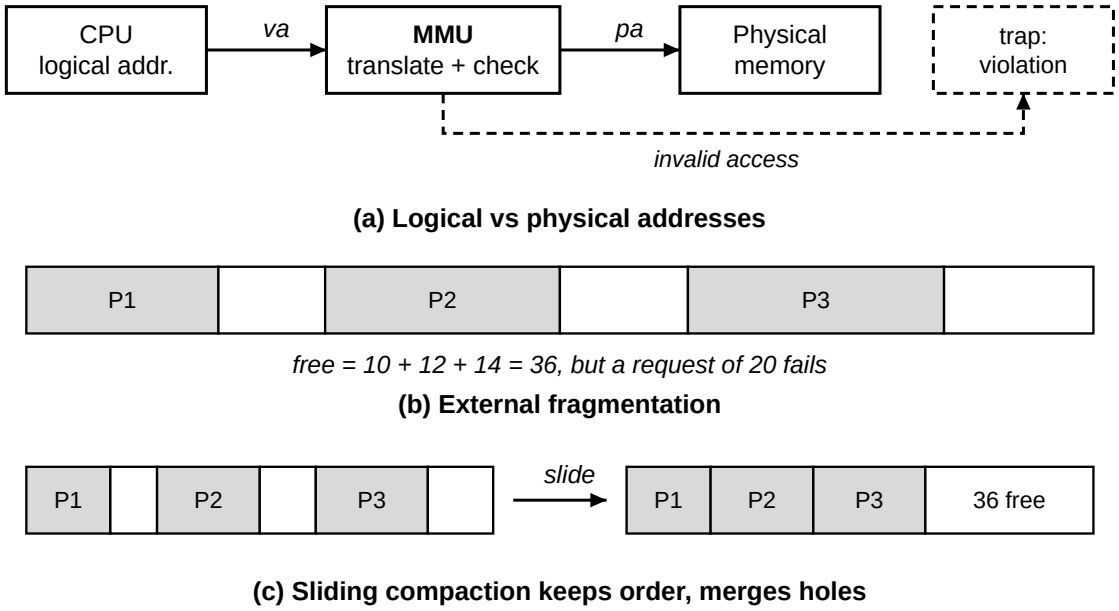


Fig. 3. Address translation, external fragmentation and compaction.

Why paging? With contiguous allocation, free space ends up scattered in small gaps. Paging avoids this: every piece is the same size, so any free slot fits any piece. The cost is a lookup table per program that records where each piece currently lives.

Paging splits logical memory into fixed-size *pages* and physical memory into equal *frames*; a page table maps one to the other (Fig. 4). Under demand paging a page is loaded only on its first use; a reference to a non-resident page is a *page fault*. When no frame is free, a replacement policy picks a victim: FIFO the oldest page, LRU the least recently used, OPT the page used furthest in the future.

Belady's anomaly—more frames, more faults—can occur with FIFO but not with stack algorithms such as LRU and OPT (Fig. 5).

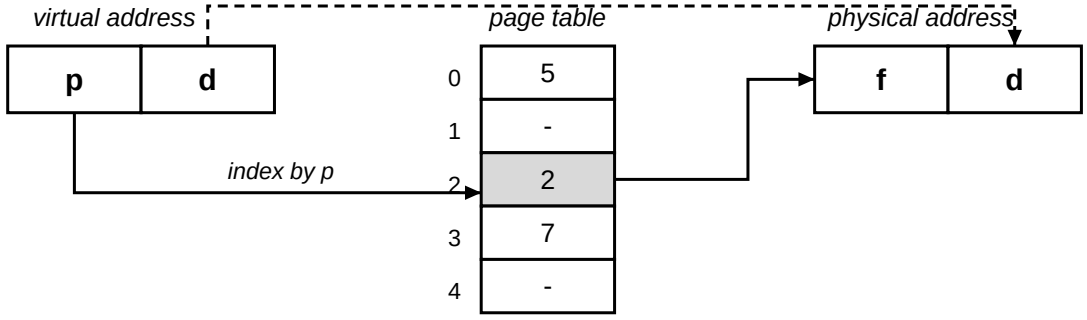


Fig. 4. Paging: the page number is replaced by a frame number.

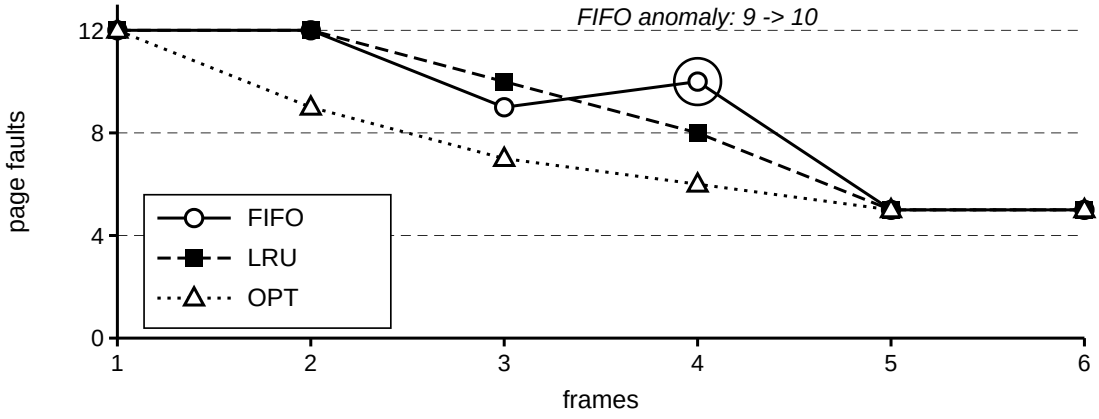


Fig. 5. Faults vs. frames on the string 1 2 3 4 1 2 5 1 2 3 4 5.

4.1 Layered Architecture

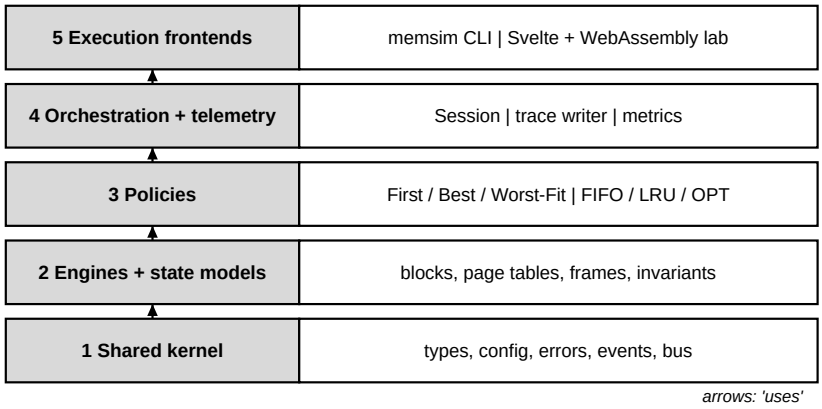


Fig. 6. Five layers; each layer uses only the ones below it.

4.2 Deterministic Core and Session Facade

The core performs no I/O and reads no wall clock; it uses logical clocks (*step*, *t*, *seq*), integer arithmetic and fixed tie-breaks (lowest address, lowest frame). Every client goes through the `Session` facade, which applies one operation, checks invariants and publishes the step's events in `seq` order (Fig. 7).

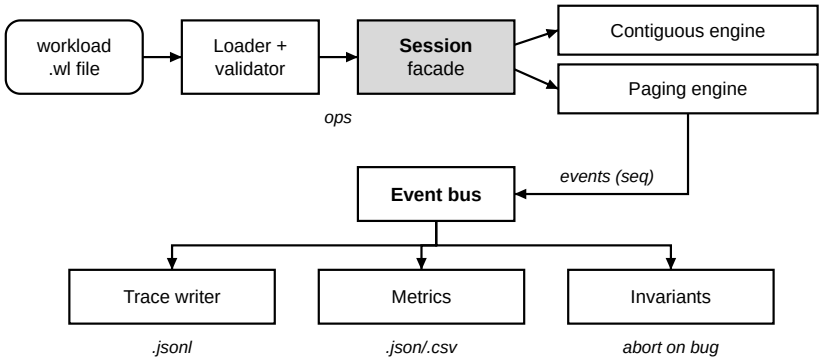


Fig. 7. End-to-end data flow of one run.

4.3 Events, Traces and WebAssembly Bridge

Each step yields events (23 types, Appendix) that the trace writer stores as JSON Lines with a snapshot after every step. The same C modules are compiled with Clang for the WASI target and loaded by the web app, so browser and CLI traces are byte-identical.

TABLE I

Architecture decision log

Decision	Reason
C17	systems language, no runtime
Address-ordered block array	simple, deterministic
Policies separate from engines	testable, cannot corrupt state
Event log as contract	one source for trace, metrics, tests
Integer-only state	identical output on every machine
Global frame pool	matches textbook examples
Plain-text workloads	no JSON parser in C

4.4 Component Workflows

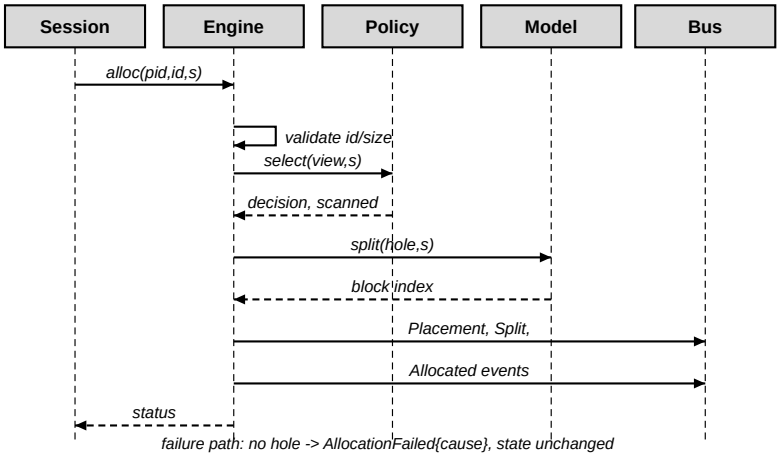


Fig. 8. Allocation sequence in the contiguous subsystem.

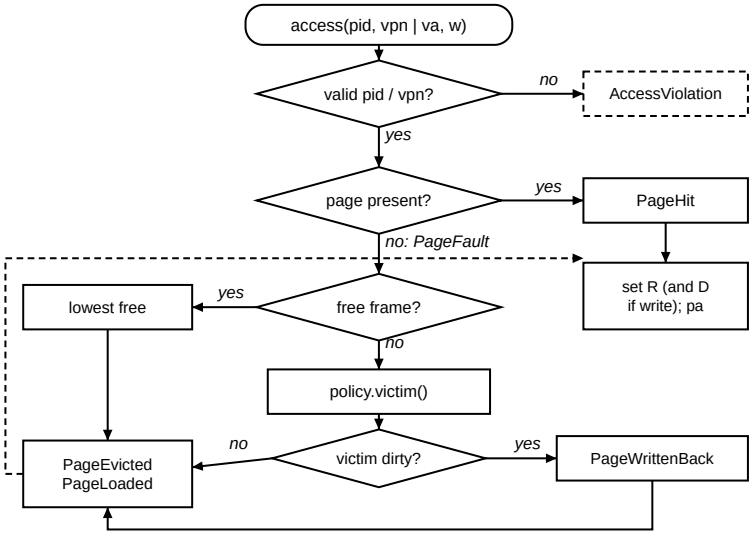


Fig. 9. Paged memory access with fault handling and replacement.

TABLE II

Module catalogue

Module	Responsibility
common	types, config, errors, events, bus, registry
contiguous	blocks, engine, FF/BF/WF, fragmentation, compaction
paging	page/frame tables, MMU, FIFO/LRU/OPT, Belady sweep
trace	JSONL writer and snapshots
workloads	loader, validator, seeded generators
metrics	every reported figure, from events
session	facade: apply, check, publish
bench	runner over workload × policy × seed
cli	memsim commands, SHA-256

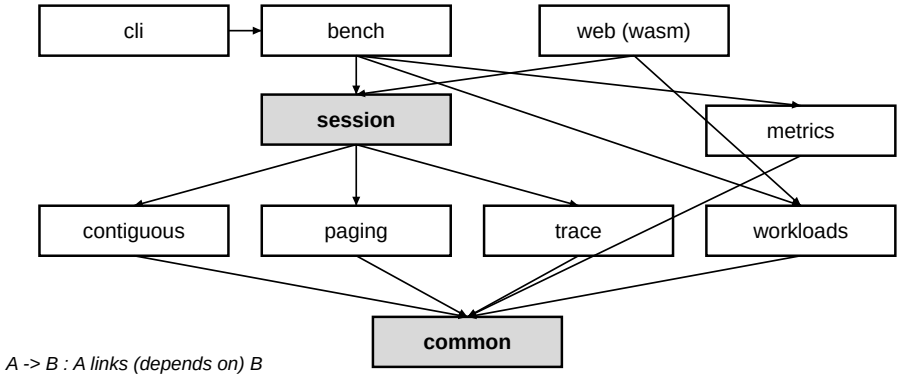


Fig. 10. Module dependency graph (acyclic, checked by check_layers.sh).

Each module exports only what others need; policies cannot modify engine data, and the core may not include `stdio.h`, `time.h` or system headers. Policies are function-pointer tables registered by name (Fig. 11), so a new policy is one file and one registry line.

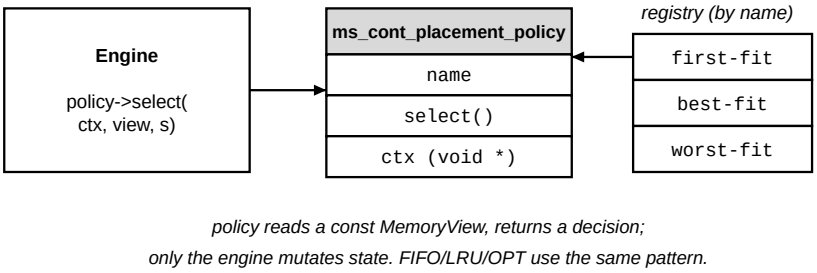


Fig. 11. Policy vtable and registry.

In plain terms, MemForge keeps a list of memory pieces in address order; each piece is either *used* (owned by a process) or *free* (a hole). Allocation cuts a hole in two; freeing joins a piece with free neighbours so holes never sit side by side.

Memory [0, M) is an address-ordered array of `ms_block` records that tile it exactly. Workloads refer to blocks by stable allocation ids, never by address, because compaction moves blocks; each id passes through the lifecycle in Fig. 12, which rules out duplicate ids and double frees.

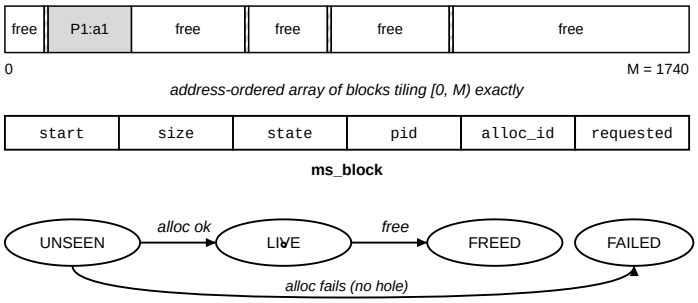


Fig. 12. Block array, block record and allocation-id lifecycle.

Algorithm 1: Block splitting

1: check the hole is free and size ≥ s
2: **if** size > s **then** insert a free block [start + s, end) after it
3: set size ← s, pid, alloc_id, requested; mark ALLOCATED
4: **return** block index

Algorithm 2: Block freeing and coalescing

1: verify the block is ALLOCATED
2: mark FREE; clear pid and alloc_id
3: **if** right neighbour is free **then** merge it
4: **if** left neighbour is free **then** merge into it

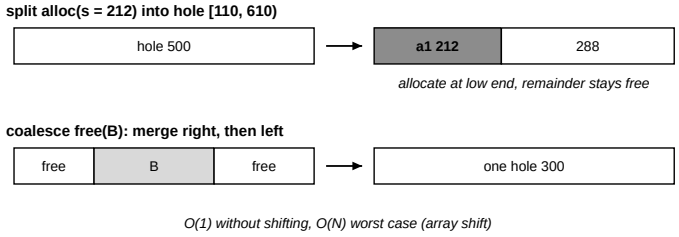


Fig. 13. Splitting a hole on allocation and coalescing on free.

Splitting and freeing cost O(1) when no block has to shift and O(N) in the worst case (array shift); both use O(1) extra space.

7. FIRST-FIT, BEST-FIT AND WORST-FIT ALGORITHMS

When several holes are big enough, which one should be used? Picture parking a car: take the *first* space that fits (First-Fit), the *tightest* space that fits (Best-Fit), or the *biggest* space (Worst-Fit). Each choice affects how usable the leftover gaps will be later.

A placement policy is a pure selector: given the holes in address order and a size *s*, it returns whether a hole was found, which one, and how many holes it *scanned* (the search-cost metric). Algorithm 1 gives all three; they differ only in the selection test.

Algorithm 3: Placement selection (FF / BF / WF)

```
1:  if view = NULL or s ≤ 0 then return (found = false, scanned = 0)
2:  best ← none
3:  for i ← 0 to h − 1 do
4:    if holes[i].size < s then continue
5:    FF: return (true, i, scanned = i + 1)
6:    BF: if best = none or size < holes[best].size then best ← i
7:    WF: if best = none or size > holes[best].size then best ← i
8:  end for
9:  return (best ≠ none, best, scanned = h)
```

Strict comparisons keep the lowest address on ties. All three run in O(h) time and O(1) space. Fig. 14 shows them on the textbook five-hole memory.

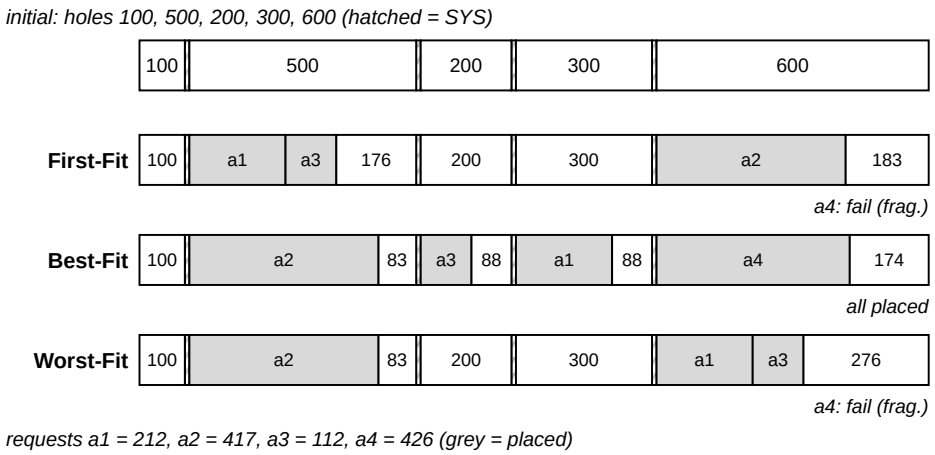


Fig. 14. Placements on the Silberschatz five-hole memory (AC-1).

Excluded: Next-Fit (a roving cursor complicates coalescing and compaction for no clear gain), size-indexed trees (O(log h) but harder to keep deterministic) and the buddy system (power-of-two sizes cause internal fragmentation).

8. FRAGMENTATION AND COMPACTION

The problem. After many allocations and frees, free memory is broken into small gaps. A new request may fail even though the *total* free memory is enough, simply because no single gap is big enough. MemForge measures how bad this is, and can fix it by *compaction*: sliding all used pieces to one end so the gaps merge into one big hole, at the cost of copying data.

TABLE III

Fragmentation metrics

Metric	Definition
total_free, largest_hole, hole_count	sum of holes; largest; number of holes
ext_frag	1 − largest_hole / total_free (0 if none free), in ppm
failure cause	FRAGMENTATION if total_free ≥ s > largest; else CAPACITY
blocks_moved, units_moved	cost of compaction

Algorithm 4: External fragmentation and failure cause (integer, ppm)

1: total ← Σ hole sizes; largest ← max hole size
2: **if** total = 0 **then** ext_frag ← 0 **else** ext_frag ← 10^6 − ppm(largest, total)
3: **on failed request** s: cause ← FRAGMENTATION **if** total ≥ s **else** CAPACITY

Example: First-Fit fails 426 with 959 free and largest hole 300, so ext_frag = 1 − 300/959 = 0.687 and the cause is FRAGMENTATION.

Algorithm 5: Sliding compaction

1: next ← 0; moves ← [ ]
2: **for each** allocated block b **in** address order **do**
3: **if** b.start ≠ next **then** append (b.id, b.start, next) to moves
4: next ← next + b.size
5: validate moves (sources match, ascending); apply; emit BlockRelocated per move
6: replace all free blocks by one hole [next, M)

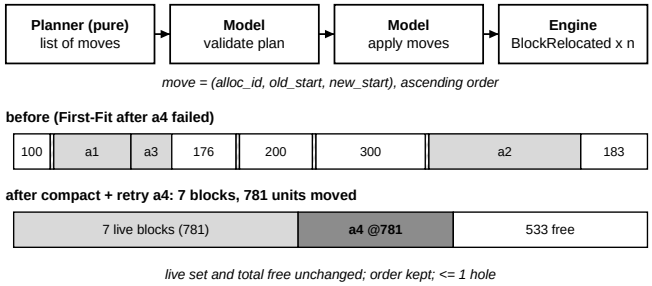


Fig. 15. Compaction pipeline and its effect on AC-2.

Tested guarantees: the live set and total free memory are unchanged, at most one hole remains, block order is preserved, and 5,000 random layouts agree with an independent oracle.

TABLE IV

AC-1: start address per request (memory 1740)

Req.	Size	FF	BF	WF
a1	212	110	830	1140
a2	417	1140	110	110
a3	112	322	620	1352
a4	426	fail	1140	fail

TABLE V

AC-1 summary metrics

Metric	FF	BF	WF
holes scanned	23	29	29
mean scanned	1.769	2.231	2.231
used / free	781 / 959	1207 / 533	781 / 959

First- and Worst-Fit fail a4 with 959 units free because the largest hole is 300 (external fragmentation). In AC-2, compaction moves 7 blocks (781 units) into one 959-unit hole [781, 1740); the retried a4 lands at 781, leaving [1207, 1740) free (Fig. 15).

Experiment E1 runs 30 seeded workloads (60/75/90% target utilisation × 10 seeds; 3,000 operations on 1,024 units; sizes 16–128; allocation probability 0.6). Figs. 16–17 plot mean values; Section 15 gives 95% confidence intervals.

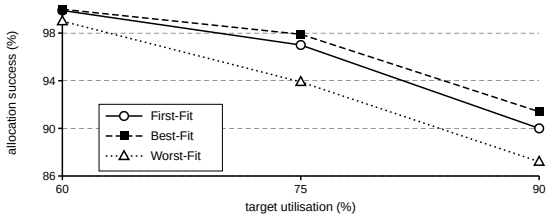


Fig. 16. E1: allocation success rate vs. target utilisation.

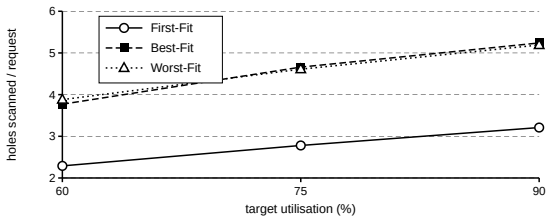


Fig. 17. E1: search cost (mean holes scanned per request).

Each program sees its own private *virtual* addresses starting at 0. Its memory is cut into pages (here 4096 bytes); RAM is cut into frames of the same size. A per-program *page table* says which frame holds each page. To read an address, the hardware splits it into "which page" and "how far into the page", looks up the frame, and keeps the offset unchanged.

The paging subsystem has four parts: MMU, per-process page tables, an inverted frame table and the paging engine (Fig. 18).

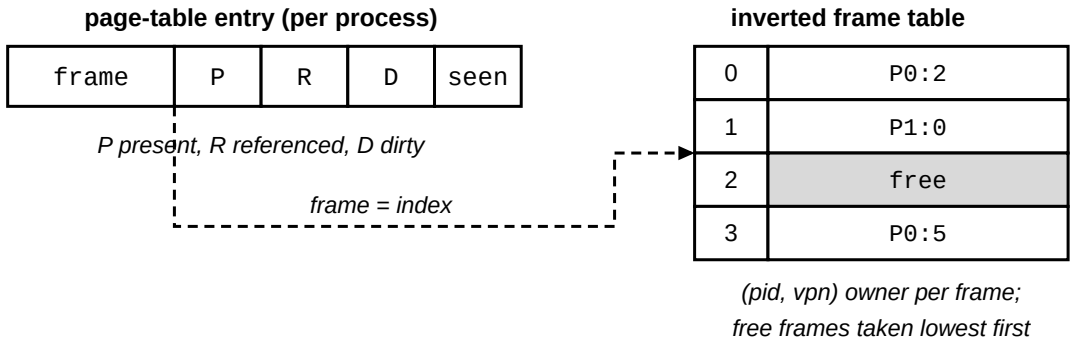


Fig. 18. Page-table entry and inverted frame table.

For a power-of-two page size S, $k = \log_2 S$; vpn = va >> k, offset = va & (S−1) and pa = (f << k) | offset (Fig. 19). The full access path, including hit, fault, eviction and write-back, is Fig. 9. Faults are *compulsory* on a page's first use and *capacity* afterwards. A hit costs O(1), a fault O(F); space is O(P + F).

Algorithm 6: Paging address translation

validate pid and vpn; **if** invalid **then** AccessViolation, **return**
refs ← refs + 1
if page present **then** PageHit
else PageFault; take lowest free frame **or** policy victim
  if victim dirty **then** PageWrittenBack; evict; load page; update frame owner
set R; **if** write **then** set D
return pa = (frame << k) | offset

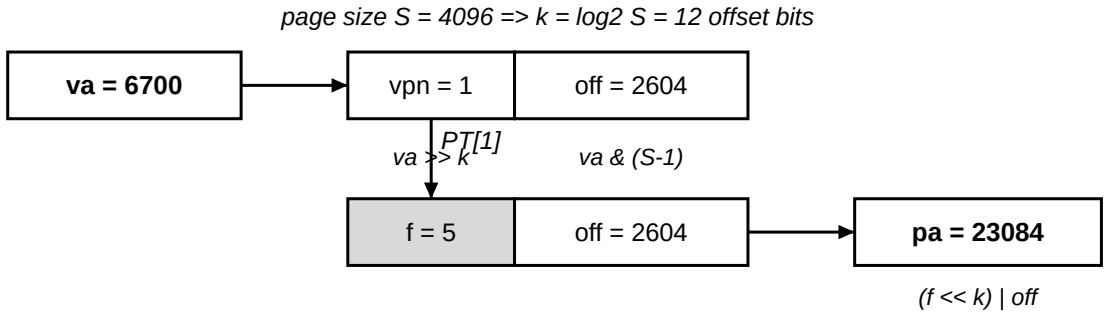


Fig. 19. Translating va = 6700 with page size 4096.

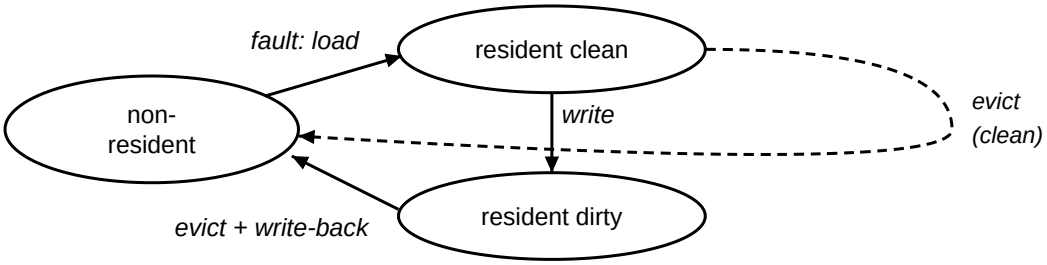


Fig. 20. Page life cycle; dirty victims are written back.

TABLE VI

Paging invariants (checked after every step)

ID	Invariant
V1	page tables and frame owners agree
V2	every frame is free or owned
V3	non-resident pages have no frame, R or D
V4	resident pages ≤ frames
V5	refs = hits + faults; write-backs ≤ evictions

TABLE VII

Negative tests (state unchanged)

Access	Condition	Result
P0, page 9	outside address space	OUT_OF_RANGE
P9, page 0	no such process	NO_SUCH_PROCESS

11. FIFO, LRU AND OPTIMAL REPLACEMENT

When a needed page is not in RAM and every frame is already full, some page must be thrown out to make room. A good choice throws out a page that will not be needed soon; a bad choice throws out a page that is needed again immediately, causing another fault. FIFO, LRU and OPT are three rules for making this choice.

A replacement policy keeps only its own bookkeeping and answers one question: which frame to evict. The engine calls its hooks in a fixed order (Fig. 21); ties go to the lowest frame.

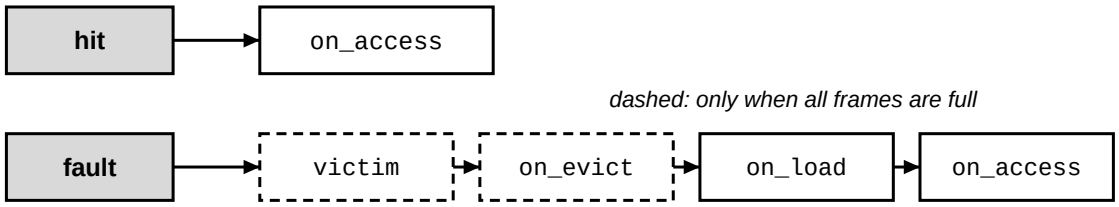


Fig. 21. Order of replacement-policy hooks on a hit and on a fault.

Algorithm 7: Victim selection for FIFO, LRU and OPT

input: full frames 0..F−1, current time t
FIFO: return frame with smallest load_time
LRU: return frame with smallest last_use
OPT: for each frame f: nu(f) ← next index > t where its page is referenced (∞ if never)
    return frame with largest nu(f)
ties: lowest frame index; hooks update load_time on load and last_use on every access

TABLE VIII

Replacement policies compared

Policy	State	Per fault	Future?	Stack?
FIFO	load time / frame	O(F)	no	no
LRU	last use / frame	O(F)	no	yes
OPT	next-use table	O(F)	yes	yes

OPT evicts the page whose next use is furthest away, which can never cause an earlier fault than any other choice (Belady, 1966); it is therefore the lower bound used to judge FIFO and LRU. Because it needs the whole reference string, the session refuses OPT in live interactive runs.

12. PAGE REPLACEMENT WORKED EXAMPLES

The grids below are what `memsim refs --table` prints for the 20-reference string of Silberschatz with 3 frames (AC-3). Shaded cells mark the frame loaded on a fault.

ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
f0	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
f1		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
f2			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
fault	**F**	**F**	**F**	**F**		**F**	**F**	**F**	**F**	**F**	**F**			**F**	**F**			**F**	**F**	**F**

Fig. 22. FIFO: 15 faults (6 compulsory, 9 capacity).

ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
f0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
f1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
f2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
fault	**F**	**F**	**F**	**F**		**F**		**F**	**F**	**F**	**F**			**F**		**F**		**F**		

Fig. 23. LRU: 12 faults (6 compulsory, 6 capacity).

ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
f0	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
f1		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
f2			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
fault	**F**	**F**	**F**	**F**		**F**		**F**			**F**			**F**				**F**		

Fig. 24. OPT: 9 faults (6 compulsory, 3 capacity).

All three evict page 7 at reference 4. They first diverge at reference 6 (page 3): FIFO evicts page 0, which is needed immediately, while LRU and OPT evict page 1.

12.1 Belady's Anomaly (AC-4)

ref	**1**	**2**	**3**	**4**	**1**	**2**	**5**	**1**	**2**	**3**	**4**	**5**
f0	1	1	1	4	4	4	5	5	5	5	5	5
f1		2	2	2	1	1	1	1	1	3	3	3
f2			3	3	3	2	2	2	2	2	4	4
fault	**F**	**F**	**F**	**F**	**F**	**F**	**F**			**F**	**F**	

Fig. 25. FIFO, 3 frames: 9 faults.

ref	**1**	**2**	**3**	**4**	**1**	**2**	**5**	**1**	**2**	**3**	**4**	**5**
f0	1	1	1	1	1	1	5	5	5	5	4	4
f1		2	2	2	2	2	2	1	1	1	1	5
f2			3	3	3	3	3	3	2	2	2	2
f3				4	4	4	4	4	4	3	3	3
fault	**F**	**F**	**F**	**F**			**F**	**F**	**F**	**F**	**F**	**F**

Fig. 26. FIFO, 4 frames: 10 faults.

With 4 frames, from the seventh reference on FIFO always evicts exactly the page needed next, so every reference faults.

TABLE IX

E4 configuration

Parameter	Value
virtual pages / page size	32 / 4096 B
references per run	2,000
working set / phase length	6 pages / 200 refs
out-of-set probability	5%
seeds / frames	10 / 1–16

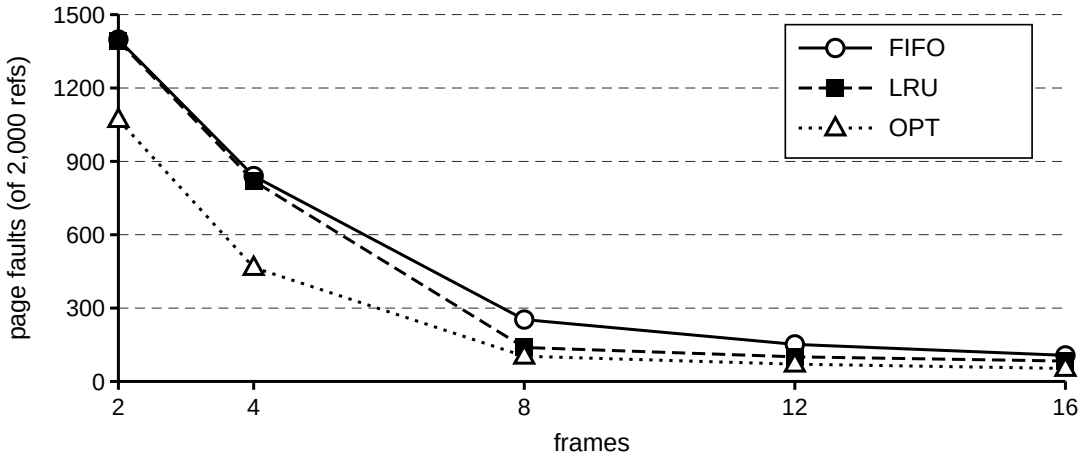


Fig. 27. E4: mean page faults vs. frames (10 seeds).

TABLE X

E4 page faults, mean ± 95% CI

Frames	FIFO	LRU	OPT
2	1398.2 ± 21.2	1393.0 ± 18.8	1070.8 ± 13.9
4	840.0 ± 15.9	819.6 ± 19.9	466.0 ± 9.3
8	253.1 ± 16.5	138.9 ± 10.1	103.2 ± 5.9
12	151.7 ± 9.7	100.7 ± 6.0	70.7 ± 4.0
16	106.7 ± 9.0	83.5 ± 5.5	53.2 ± 3.2

Smooth locality hides non-monotonic behaviour, but the AC-4 sweep exposes it: FIFO gives 12, 12, 9, 10, 5, 5 faults for 1–6 frames while LRU and OPT never rise (Fig. 5). In practice OPT is unimplementable, FIFO is simple but anomalous, and LRU is strong under locality but costly, which is why real systems approximate it with Clock.

A *workload* is simply a script of memory requests (allocate, free, access a page) that is fed to the simulator. A *metric* is a number measured while it runs, such as how many requests succeeded or how many page faults occurred. Using the same workload for every policy makes the comparison fair.

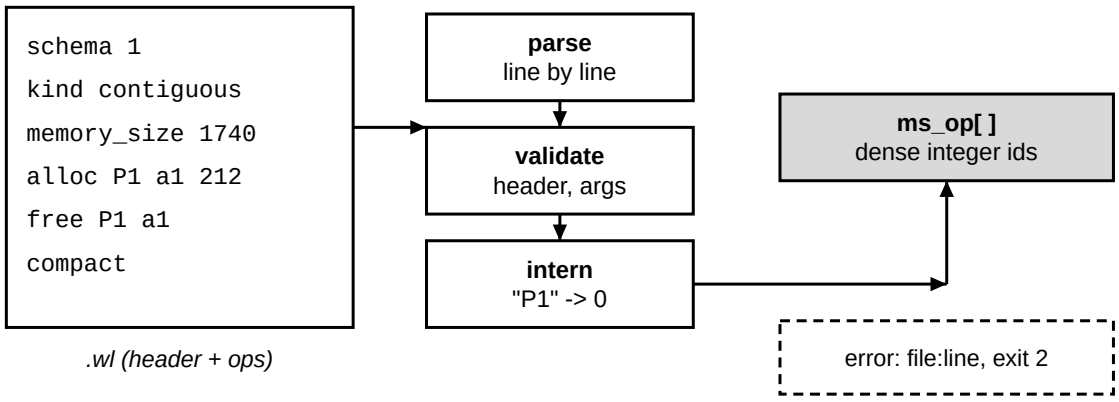


Fig. 28. Workload file format and loader pipeline.

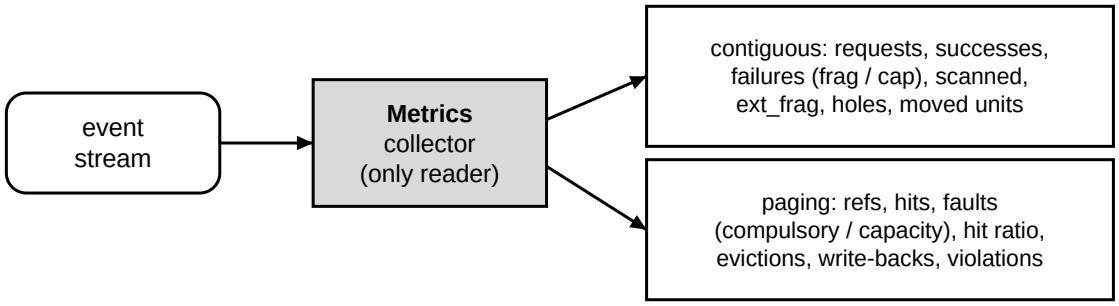


Fig. 29. One collector derives every metric from the event stream.

Each benchmark below is reported in the same four parts: **setup**, **result table**, **chart** and **finding**. Values are mean ± 95% CI over 10 seeds.

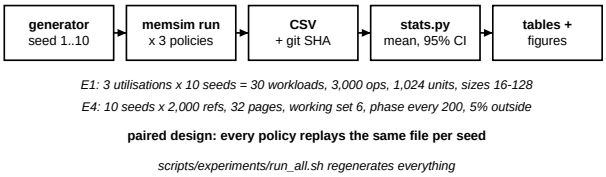


Fig. 30. Experiment pipeline (paired, seeded, reproducible).

15.1 Benchmark E1: Placement Policies under Memory Pressure

Setup. 30 workloads (60%, 75%, 90% target utilisation × 10 seeds), 3,000 operations each on 1,024 units, request sizes 16–128, no compaction.

TABLE XI

E1 results, mean ± 95% CI

Util.	Policy	Success %	Frag. fail.	Scanned	Ext. frag.
60%	FF	99.9 ± 0.0	1.2 ± 0.7	2.29	0.359
60%	BF	100.0 ± 0.0	0.3 ± 0.3	3.77	0.322
60%	WF	99.0 ± 0.2	14.3 ± 3.1	3.88	0.546
75%	FF	97.0 ± 0.3	44.6 ± 4.7	2.78	0.460
75%	BF	97.9 ± 0.2	32.0 ± 2.8	4.66	0.419
75%	WF	93.9 ± 0.5	92.4 ± 6.8	4.61	0.597
90%	FF	90.0 ± 0.6	148.6 ± 8.6	3.21	0.516
90%	BF	91.4 ± 0.4	127.7 ± 7.0	5.24	0.473
90%	WF	87.2 ± 0.7	192.0 ± 11.1	5.19	0.629

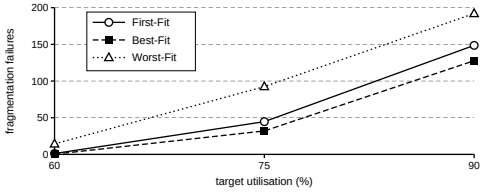


Fig. 31. E1: requests failed by fragmentation despite enough free memory.

TABLE XII

E1 findings

Aspect	Finding
Quality	Best-Fit: highest success, lowest fragmentation at every level
Cost	First-Fit scans 2.3–3.2 holes vs. 3.8–5.2 for the others
Pressure	at 90%: 149 (FF), 128 (BF), 192 (WF) fragmentation failures
Reproducibility	every cell regenerates from generator + seed + git revision

15.2 Benchmark E4: Page Faults against Frames

Setup. 10 seeds × 2,000 references, 32 pages, working set 6, phase change every 200 references; FIFO, LRU, OPT at 2–16 frames. **Result:** Table X and Fig. 27 (Section 13).

TABLE XIII

E4 findings

Aspect	Finding
Ordering	OPT < LRU < FIFO at every frame count
At 8 frames	103.2 (OPT), 138.9 (LRU), 253.1 (FIFO) faults
Anomaly	only FIFO, on the AC-4 string: 9 → 10 faults

Testing answers one question: does the simulator give the right answers? We check it against textbook examples whose answers are known, check internal consistency rules after every single step, run it twice to confirm identical output, and run memory-error detectors over the C code.

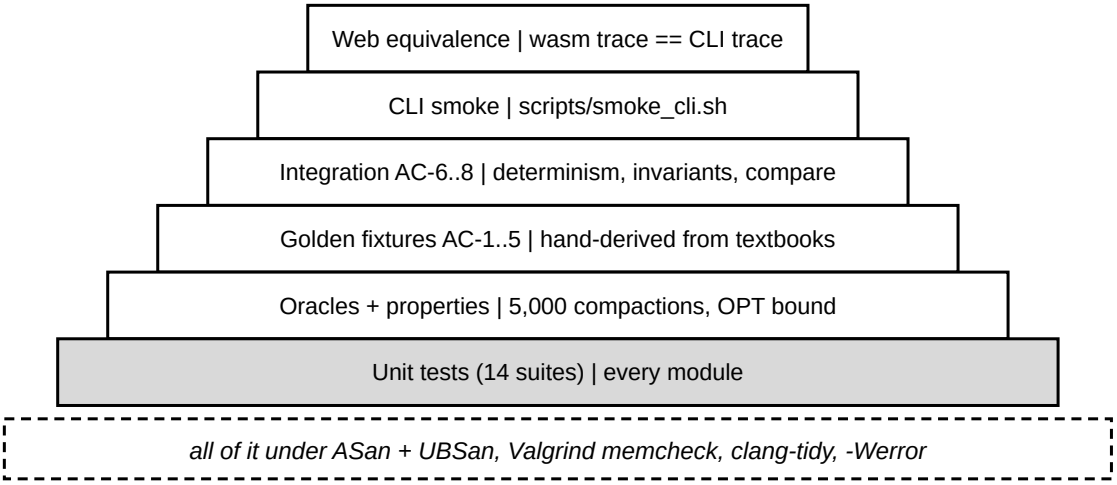


Fig. 32. Independent test layers; a bug missed by one is caught by another.

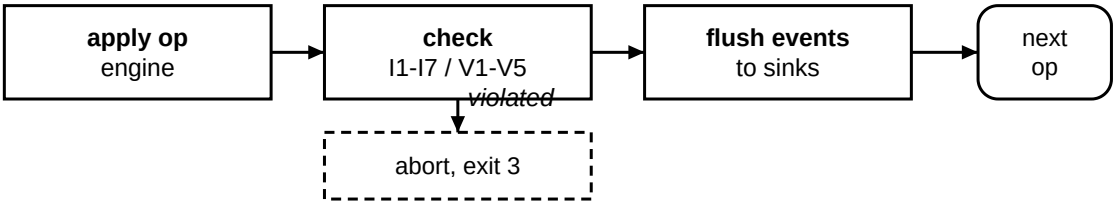


Fig. 33. Checked mode: invariants after every operation.

Randomised oracles: 5,000 compactions match a unit-by-unit oracle; no policy beats OPT; LRU and OPT never show Belady's anomaly; every fixture run twice gives equal SHA-256 trace digests (AC-6). Expected values are written into each test before it is compared with the engine, taken from the Silberschatz textbook or from Belady's original paper.

TABLE XIV

Acceptance criteria (verify_report.sh)

ID	Scenario	Expected	Result
AC-1	five-hole placement	FF/WF fail a4, BF places all	PASS
AC-2	compaction + retry	7 blocks, 781 units; [1207,1740)	PASS
AC-3	20 refs, 3 frames	FIFO 15, LRU 12, OPT 9	PASS
AC-4	Belady sweep	FIFO 12,12,9,10,5,5	PASS
AC-5	negative tests	3 rejections, 2 violations	PASS
AC-6	determinism	6/6 traces identical	PASS
AC-7	invariants	17/17 suites	PASS
AC-8	compare table	shows fail FRAGMENTATION	PASS

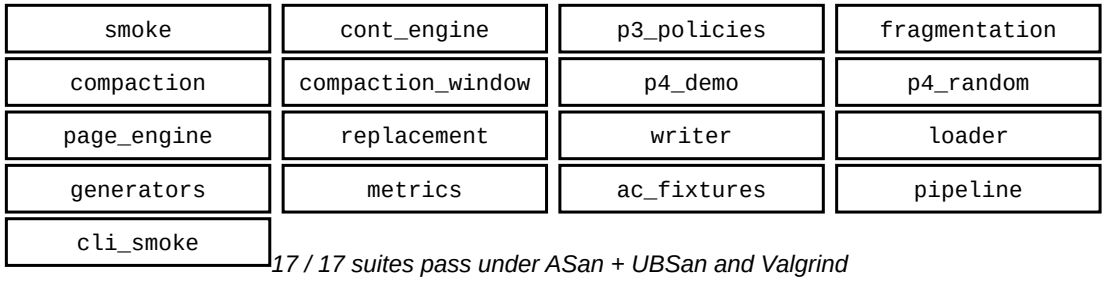


Fig. 34. Automated test suites (all passing).

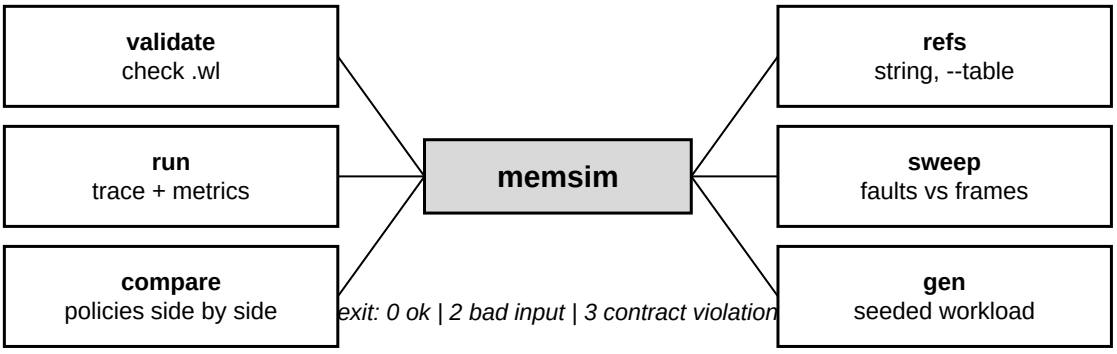


Fig. 35. The memsim command-line interface.

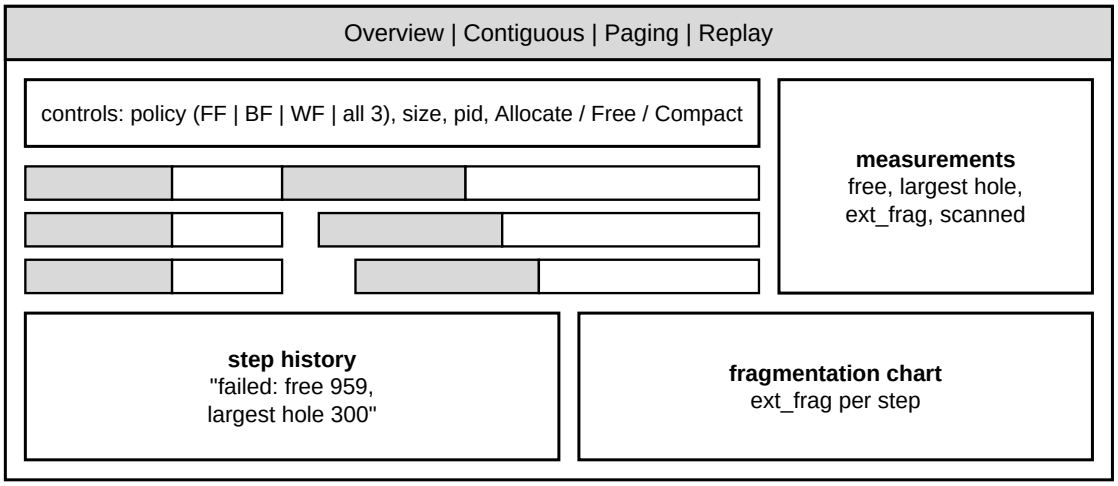


Fig. 36. Layout of the Contiguous page of the web laboratory.

The lab has four pages: Overview (live run and six classic scenarios), Contiguous, Paging (frames grid, page tables, translation, Belady curve) and Replay (step through any `memsim run --trace` file). Neither interface contains allocation or replacement logic.

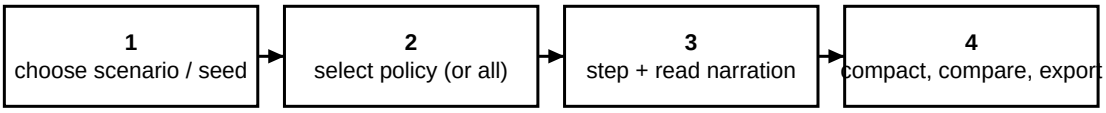


Fig. 37. Typical workflow in the lab.

Unedited terminal output (commit ab5d17d).

```
$ memsim compare ac1_silberschatz.wl \
    --policies first-fit,best-fit,worst-fit
id  size  first-fit           best-fit  worst-fit
a1  212   110                 830       1140
a2  417   1140                110       110
a3  112   322                 620       1352
a4  426   fail FRAGMENTATION  1140      fail FRAGMENTATION
```

```
$ memsim run ac2_compact.wl --policy first-fit
compactions 1   blocks moved 7   units moved 781
ext frag final 0.000000   hole count final 1
total free final 533   largest hole final 533
```

```
$ memsim sweep ac4_belady.wl --policy fifo --frames 1..6
1 12
2 12
3 9
4 10 *
5 5
6 5
```

```
$ memsim run ac5_contiguous.wl  -> rejections 3
$ memsim run ac5_paging.wl      -> violations 2
access P0 @6700 -> AddressTranslated vpn 1,
    offset 2604, frame 1, pa 6700
run summary: refs 4 hits 1 faults 3 evictions 1
```

```
$ memsim gen alloc-uniform --seed 7 -o w.wl
schema 1 / kind contiguous / memory_size 1024 /
gen_name alloc-uniform / gen_seed 7 / gen_ops 2000
```

20. REPOSITORY STRUCTURE AND DEVELOPMENT CONVENTIONS

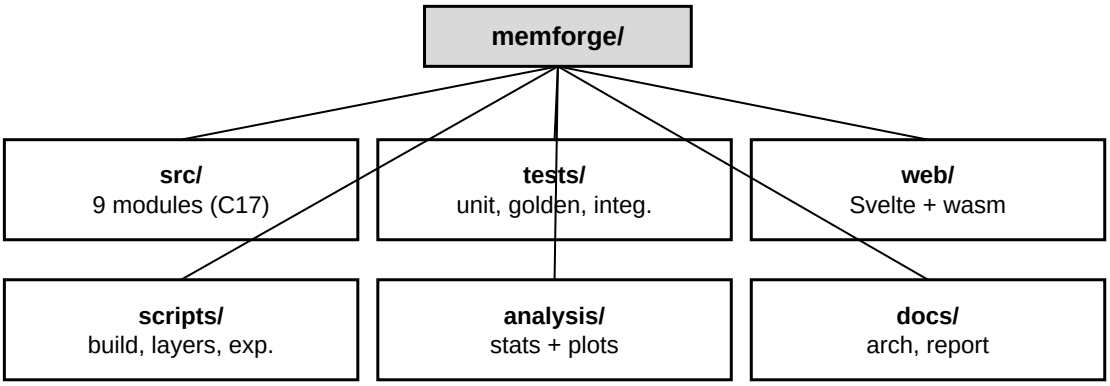


Fig. 38. Top-level repository layout.

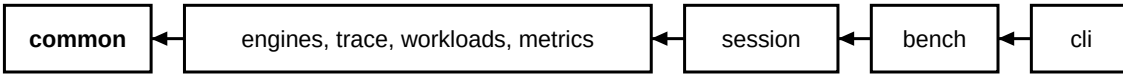


Fig. 39. Allowed include direction between module layers.

TABLE XV

Conventions and ownership

Rule	Practice
naming	prefix ms_ / MS_, e.g. ms_cont_engine_alloc
format	clang-format, .editorconfig
ownership	CODEOWNERS, roles P1–P10
interfaces	change via RFC, lead approves
commits	pre-commit: format + layering

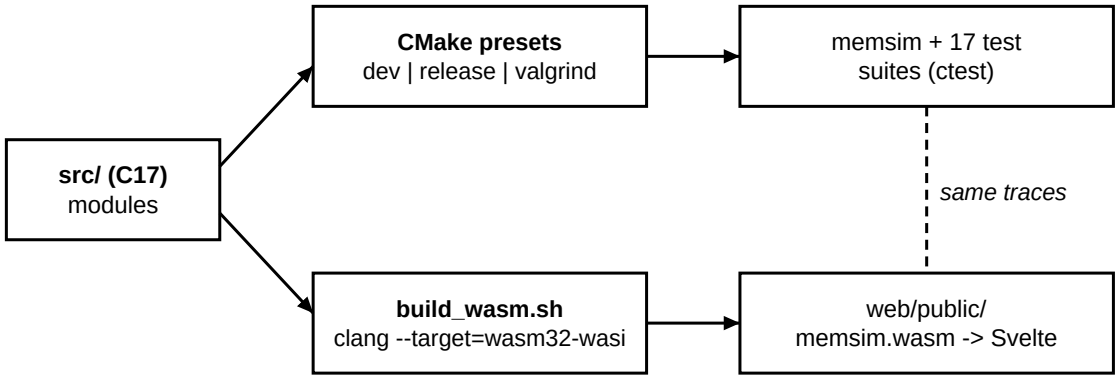


Fig. 40. Build orchestration: native presets and the WebAssembly build.

TABLE XVI

CMake presets

Preset	Configuration	Use
dev	Debug + ASan + UBSan	daily work, main tests
release	-O2	experiments, binary
valgrind	Debug	memcheck on every test

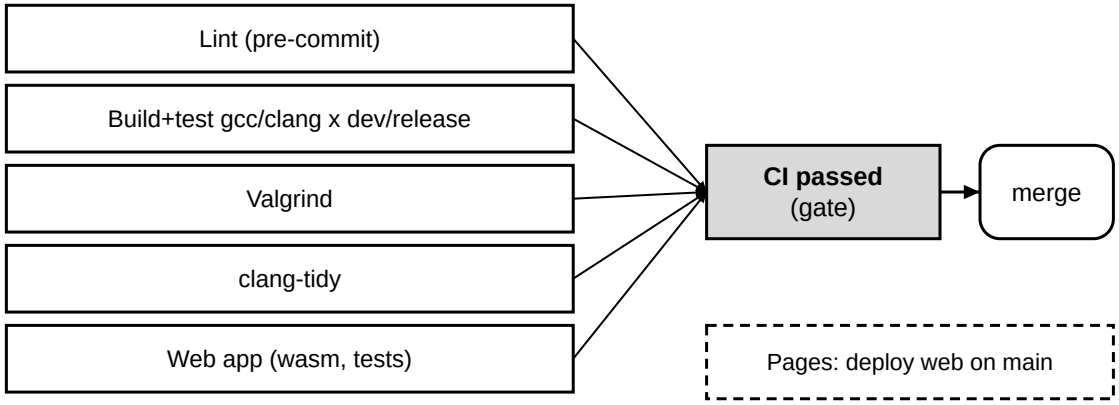
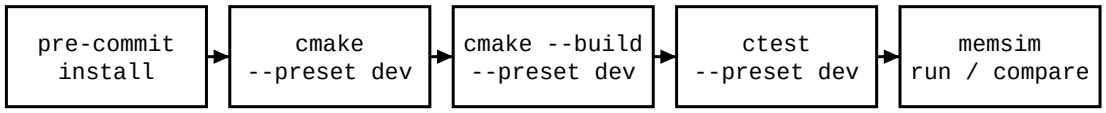


Fig. 41. GitHub Actions jobs and the single merge gate.

All code builds as C17 with `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion`, and with `-Werror` in CI.

22. SETUP INSTRUCTIONS AND DOCUMENTATION

Requirements: Linux or macOS (Windows via WSL), GCC or Clang, CMake ≥ 3.25, Ninja, pre-commit; Python 3 + matplotlib for experiments; Node.js and Clang/lld/wasi-libc for the web app.



web: build_wasm.sh -> npm ci -> npm run dev | npm test

Fig. 42. Build, test and run sequence.

```
$ cmake --preset release && cmake --build --preset release
$ bash scripts/experiments/run_all.sh
$ bash scripts/report/verify_report.sh
```

TABLE XVII

Documentation

Document	Content
README.md	overview, quick start, commands
docs/ARCHITECTURE.md	requirements, design, ADRs
docs/report/	report, tables, figures
analysis/README.md	statistics and plots
.github/	PR, bug and RFC templates

A hosted copy of the lab is at memforge.aryagupta-dev.xyz.

23. CONCLUSION AND FUTURE SCOPE

MemForge meets its goals: a deterministic, modular C17 simulator that reproduces the textbook examples exactly and compares policies fairly. Best-Fit gives the best success and lowest fragmentation, First-Fit is cheapest, Worst-Fit is worst on both; OPT < LRU < FIFO at every frame count, and only FIFO shows Belady's anomaly.

TABLE XVIII

Goals and evidence

Area	Delivered	Evidence
Contiguous	FF/BF/WF, split, coalesce, compaction	AC-1, AC-2, E1
Paging	translation, FIFO/LRU/OPT, sweep	AC-3–5, E4
Determinism	byte-identical traces	AC-6
Correctness	invariants, oracles	AC-7, 17/17
Delivery	CLI + wasm from one code base	web equivalence test

Lessons: determinism must be designed in (integer state, logical clocks, tie-breaks); separating decisions from mutation made policies trivially testable; compiling the real engine to WebAssembly removed demo drift; automated layering checks caught boundary violations early.

Future work: Next-Fit, buddy allocation, alignment; Clock, LFU, a TLB and two-level tables; local replacement; Knuth's fifty-percent rule; real program traces; fuzzing and a coverage gate.

TABLE XIX

Complete AC-1 comparison metrics

Metric	FF	BF	WF
requests / successes	13 / 12	13 / 13	13 / 12
failures (frag.)	1	0	1
frees	5	5	5
scanned total / mean	23 / 1.769	29 / 2.231	29 / 2.231
ext_frag final / mean	0.687 / 0.246	0.674 / 0.212	0.687 / 0.245
holes / free / largest	5 / 959 / 300	5 / 533 / 174	5 / 959 / 300
used / live blocks	781 / 7	1207 / 8	781 / 7

TABLE XX

Paging metrics for the textbook strings

Run	Hits	Faults	Comp.	Cap.	Hit ratio	Evict.
AC-3 FIFO	5	15	6	9	0.250	12
AC-3 LRU	8	12	6	6	0.400	9
AC-3 OPT	11	9	6	3	0.550	6
AC-4 FIFO	3	9	5	4	0.250	6
AC-4 LRU	2	10	5	5	0.167	7
AC-4 OPT	5	7	5	2	0.417	4

```
{"seq":40,"step":18,"type":"PlacementDecided",
 "data":{"policy":"first-fit","requested":426,
  "hole":null,"scanned":5}}
{"seq":41,"step":18,"type":"AllocationFailed",
 "data":{"pid":"P4","id":"a4","requested":426,
  "cause":"FRAGMENTATION","total_free":959,
  "largest_hole":300}}
```

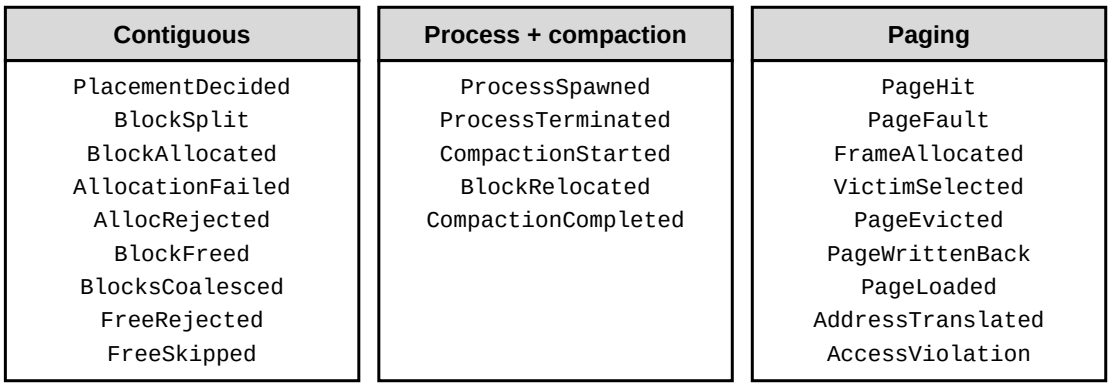


Fig. 43. Event catalogue (23 types).